

CFRM 521/421 Machine Learning for Finance

Course Final Project

Project Title: A Machine Learning Approach to Next Day S&P 500 Direction Forecasting

Group Members:

- Anna Chen (email: annac30@uw.edu, Algorithm: Logistic Regression)
- Yue Lu (email: yueyuee@uw.edu, Algorithm: KNN)
- David Zhang (email: minghz6@uw.edu, Algorithm: Random Forest)
- Jiayuan Li (email: jli72@uw.edu, Algorithm: SVM)

Date: 06/07/2026

1. Introduction

1.1 Problem Statement

The S&P 500 is one of the most widely followed equity benchmarks in the world, and SPY captures the aggregate daily performance of large-cap U.S. equities across sectors. Whether SPY's price rises or falls on a given trading day encodes a great deal of information about market conditions, and the ability to predict that direction has long attracted the attention of both academic researchers and financial practitioners.

This project treats next-day SPY return direction prediction as a binary classification problem. Using only information available at the close of day t , each model predicts whether SPY's return on day $t+1$ will be positive or negative. This constraint is deliberate: every feature is lagged so that it reflects past observations only, preventing look-ahead bias from distorting the evaluation (Rodriguez et al., 2024).

Daily equity returns are noisy. The signal-to-noise ratio in short-horizon return prediction is notoriously low, and even modest improvements over a naive baseline may carry economic significance (Fama, 1970). We compare several standard machine learning classifiers under a shared chronological train-validation-test split to assess whether any method achieves stable predictive power over a ten-year sample (Jiao and Jakubowicz, 2017). The question we ask is empirical: does any model reliably outperform a naive benchmark that always predicts the majority class?

1.2 Project Objective

This project compares four machine learning classifiers: Logistic Regression, K-Nearest Neighbors (KNN), Random Forest, and Support Vector Machine (SVM) by following the

experimental framework for supervised classification on tabular data (Géron, 2022). Each model trains on the same 21 engineered features and predicts the same binary target, enabling a controlled comparison of model architectures under identical data conditions.

The feature set spans five categories. SPY return and lag features capture short-term momentum and mean-reversion patterns in the index itself. Moving-average ratio features measure how far SPY's price currently deviates from its 5-day, 10-day, and 20-day rolling averages. Rolling volatility features characterize recent market uncertainty using 5-day and 20-day standard deviations of daily returns. Market and macroeconomic indicators connect daily equity behavior to broader financial conditions: VIX level, VIX return, the 10-year Treasury yield (^TNX), and the Federal Funds Rate each reflect a distinct dimension of the macroeconomic environment. Sector ETF returns from XLK, XLF, and XLY contribute cross-sectional signals about how technology, financial, and consumer discretionary stocks move relative to the broader market on any given day. The Relative Strength Index (RSI) and Moving Average Convergence/Divergence (MACD) indicator add standard technical signals. All equity and index data are sourced from Yahoo Finance (2024), and the Federal Funds Rate series is obtained from FRED (Federal Reserve Bank of St. Louis, 2024).

Data spans January 2015 through December 2024 and is split strictly by calendar date. The training set covers 2015 through 2020, providing approximately 1,500 observations for model fitting. The validation set covers 2021 through 2022 and guides hyperparameter selection exclusively. The test set covers 2023 through 2024 and is evaluated exactly once, after all modeling decisions are finalized. No shuffling is applied at any stage. This design ensures that future observations never influence the training of any model, which reflects the conditions a practitioner would face when deploying such a classifier in real time.

2. Data Description

2.1 Data Source

This project draws on four data sources, all freely accessible:

Table 1. Data Sources and Access Methods

Data	Tickers	Source	Access
SPY adjusted close & volume	SPY	Yahoo Finance	<code>yfinance</code> (batch download)
Sector ETFs adjusted close	XLK, XLF, XLY	Yahoo Finance	<code>yfinance</code> (batch download)
Market indicators	^VIX, ^TNX	Yahoo Finance	<code>yfinance</code> (batch download)
Federal Funds Effective Rate	FEDFUNDS	FRED	local CSV / FRED URL fallback

All equity and index series are downloaded in a single `yf.download()` call. The FEDFUNDS series is loaded from a local CSV file (`FEDFUNDS.csv`); if the file is not found, the code automatically fetches it from the FRED public URL. All series are merged on trading date; the monthly FEDFUNDS series is forward-filled to align with daily equity data.

Note: XLY (Consumer Discretionary) replaces XLC (Communication Services) because XLC did not launch until June 2018, which would have eliminated three years of training data. ^TNX (10-Year Treasury Constant Maturity Yield) from Yahoo Finance is used as the long-term interest rate proxy, replacing a separate FRED download for DGS10.

2.2 Data Structure

- **Unit of observation:** one U.S. equity trading day.
- **Sampling frequency:** daily.
- **Time period:** January 2, 2015 – December 31, 2024 (approximately 2,516 raw trading days for SPY).
- **Final sample size:** approximately 2,470 observations after dropping NaN rows introduced by rolling-window calculations (20-day window) and the last row lost to the forward-shifted target.

2.3 Target Variable

The response variable Y_t is a binary indicator of the **next-day return direction** of SPY:

$$Y_t = \begin{cases} 1 & \text{if } r_{t+1} > 0 \quad (\text{market up}) \\ 0 & \text{if } r_{t+1} \leq 0 \quad (\text{market down or flat}) \end{cases}$$

where $r_{t+1} = \frac{P_{t+1} - P_t}{P_t}$ is the simple return on day $t + 1$. All feature values are observed on or before day t , so there is no look-ahead bias.

2.4 Features

The final feature set contains **21 predictors** grouped into five categories. All lagged or rolling features carry an explicit `.shift(1)` to ensure only information available at the close of day t is used.

Table 2. SPY Return Features

Feature	Definition
<code>spy_return</code>	SPY simple return on day t
<code>spy_return_lag_1</code>	SPY return on day $t - 1$
<code>spy_return_lag_2</code>	SPY return on day $t - 2$
<code>spy_return_lag_5</code>	SPY return on day $t - 5$

Table 3. Trend Features (MA Ratio)

Feature	Definition
<code>spy_ma_5_ratio</code>	$(P_t / \text{MA}_5 - 1)$, lagged 1 day — % deviation from 5-day MA

Feature	Definition
spy_ma_10_ratio	$(P_t/\text{MA}_{10} - 1)$, lagged 1 day
spy_ma_20_ratio	$(P_t/\text{MA}_{20} - 1)$, lagged 1 day

Table 4. Volatility Features

Feature	Definition
spy_vol_5	5-day rolling std of SPY daily returns, lagged 1 day
spy_vol_20	20-day rolling std of SPY daily returns, lagged 1 day

Table 5. Market and Macro Features

Feature	Definition
spy_volume_chg	SPY trading volume % change (day-over-day), lagged 1 day
vix_level	CBOE VIX index level, lagged 1 day
vix_return	VIX daily return, lagged 1 day
ten_year_yield	10-Year Treasury yield (^TNX), lagged 1 day
fed_funds_rate	Federal Funds Effective Rate (%), lagged 1 day
xlk_return	XLK (Technology) daily return, lagged 1 day
xlf_return	XLF (Financials) daily return, lagged 1 day
xly_return	XLY (Consumer Discretionary) daily return, lagged 1 day

Table 6. Technical Indicators

Feature	Definition
rsi_14	14-day RSI of SPY close, lagged 1 day
macd	MACD line (12-day EMA – 26-day EMA), lagged 1 day
macd_signal	9-day EMA of MACD line, lagged 1 day
macd_hist	MACD histogram (MACD – signal), lagged 1 day

2.5 Data Cleaning and Preprocessing

Missing values

- FEDFUNDS is a monthly series; it is forward-filled to daily frequency before merging.
- After constructing all features, any row containing `NaN` or `±inf` is dropped. The main sources of NaN are: (1) the 20-day rolling window (first ~19 rows), (2) the MACD slow EMA warm-up (first ~25 rows), and (3) the last row dropped by the forward-shifted target.

Outlier handling

- No observations are removed as outliers. Extreme return days (e.g., COVID crash in March 2020) are genuine market events and are retained intentionally to test model robustness.

Scaling / normalization

- MA ratio features are already scale-free (unit: fraction of price). Return-based features are naturally small decimals.
- Models sensitive to feature scale (Logistic Regression, SVM, KNN) apply `StandardScaler` fitted **only on the training set**; the same transformation is applied to validation and test sets to prevent data leakage.
- Random Forest is scale-invariant and does not require standardization.

Feature engineering notes

- Moving averages are expressed as ratios rather than raw price levels, making features comparable across the full 10-year period despite SPY price appreciation.
- Volume is converted to percentage change to remove the long-run upward trend in raw volume.

Table 7. Train / Validation / Test Split

Partition	Date Range	Approx. Observations
Training	2015-01-02 – 2020-12-31	~1,500
Validation	2021-01-01 – 2022-12-31	~504
Test	2023-01-01 – 2024-12-31	~504

No shuffling is applied at any stage. Hyperparameters are tuned on the validation set only; the test set is held out and evaluated exactly once.

Table 8. Dataset Summary after Feature Construction and Cleaning

	Item	Value
0	Observations	2481
1	Features	21
2	Start Date	2015-02-23
3	End Date	2024-12-31
4	Up Days (target = 1)	1354
5	Down / Flat Days (target = 0)	1127

3. Exploratory Data Analysis

In this section, we examine the final modeling dataset before fitting classification models. The main goals are to summarize the variables, check the target class balance, visualize market conditions over time, and inspect relationships among the predictors.

Because the target variable is the next-day direction of SPY, it is important to compare the number of up and down days. If one class is much more common than the other, accuracy alone may be misleading. We also examine SPY returns, volatility, VIX, interest rate variables, and sector ETF returns to understand the financial environment covered by the sample.

3.1 Target Class Balance

Table 9. Target Class Balance

	Count	Proportion
Down / Flat (0)	1127	0.454300
Up (1)	1354	0.545700

3.2 Selected Features

Table 10. Descriptive Statistics for Selected Features

	Mean	Std Dev	Min	Median	Max
spy_return	0.000500	0.011100	-0.109400	0.000600	0.090600
spy_return_lag_1	0.000500	0.011100	-0.109400	0.000600	0.090600
spy_vol_5	0.008700	0.007200	0.000400	0.007000	0.088300
spy_vol_20	0.009300	0.006200	0.002000	0.007600	0.058700
spy_volume_chg	0.053500	0.359200	-0.687600	-0.004600	2.360000
vix_level	18.271500	7.329500	9.140000	16.270000	82.690000
vix_return	0.003400	0.086400	-0.281600	-0.007200	1.156000
ten_year_yield	2.484600	1.059800	0.499000	2.327000	4.988000
fed_funds_rate	1.803200	1.867900	0.050000	1.160000	5.330000
xlk_return	0.000800	0.014700	-0.138100	0.001200	0.117300
xlf_return	0.000500	0.014000	-0.137100	0.000700	0.131600
xly_return	0.000600	0.013400	-0.126700	0.001200	0.093800
rsi_14	56.303600	11.479500	16.996700	57.795400	87.191700
macd	1.176700	3.446900	-21.555600	1.465800	8.928300
macd_hist	0.001200	1.118200	-5.604900	0.030800	5.595600

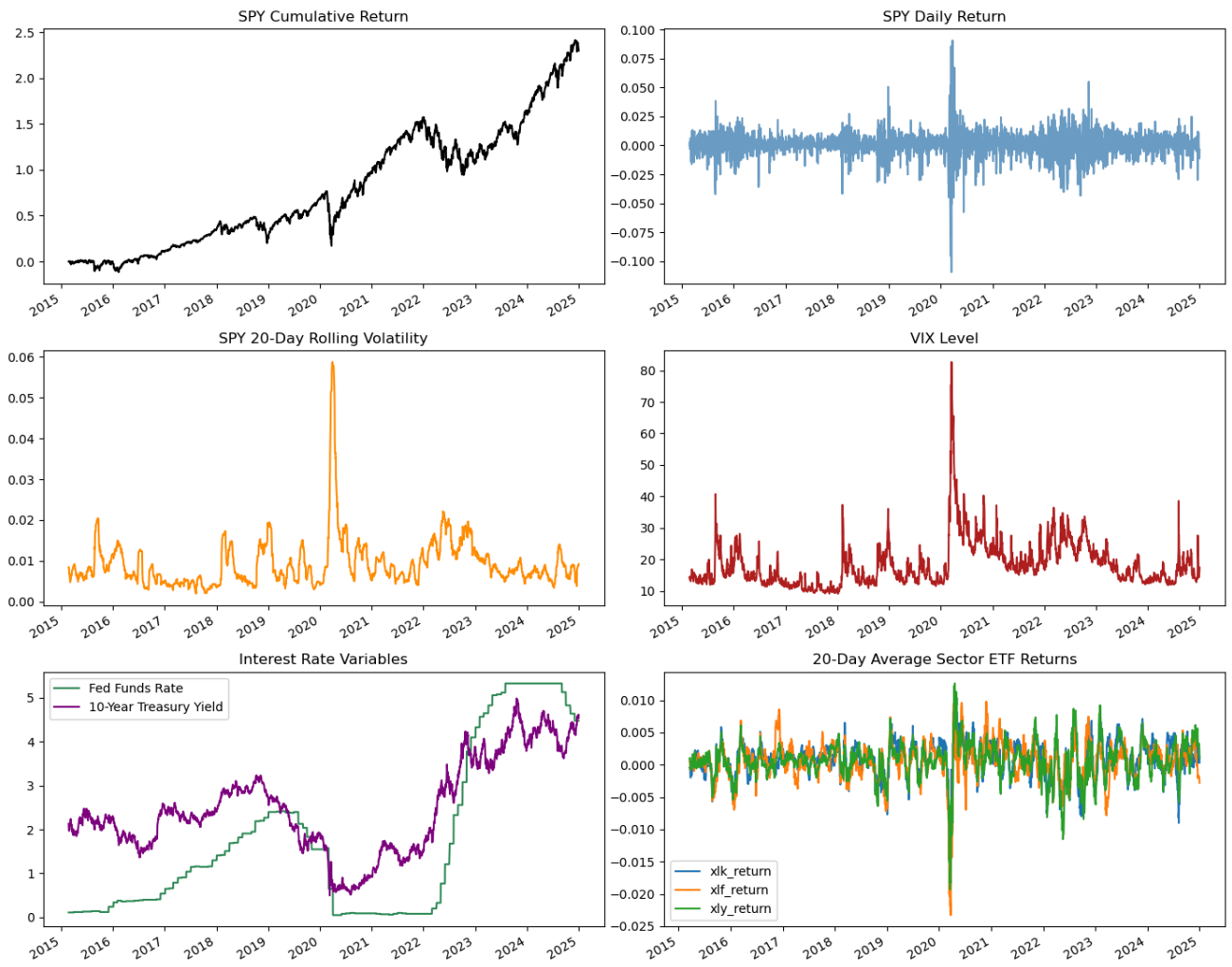


Figure 1. Time-series summary of SPY returns, volatility, VIX, interest rates, and sector ETF returns.

3.3 Feature Correlation

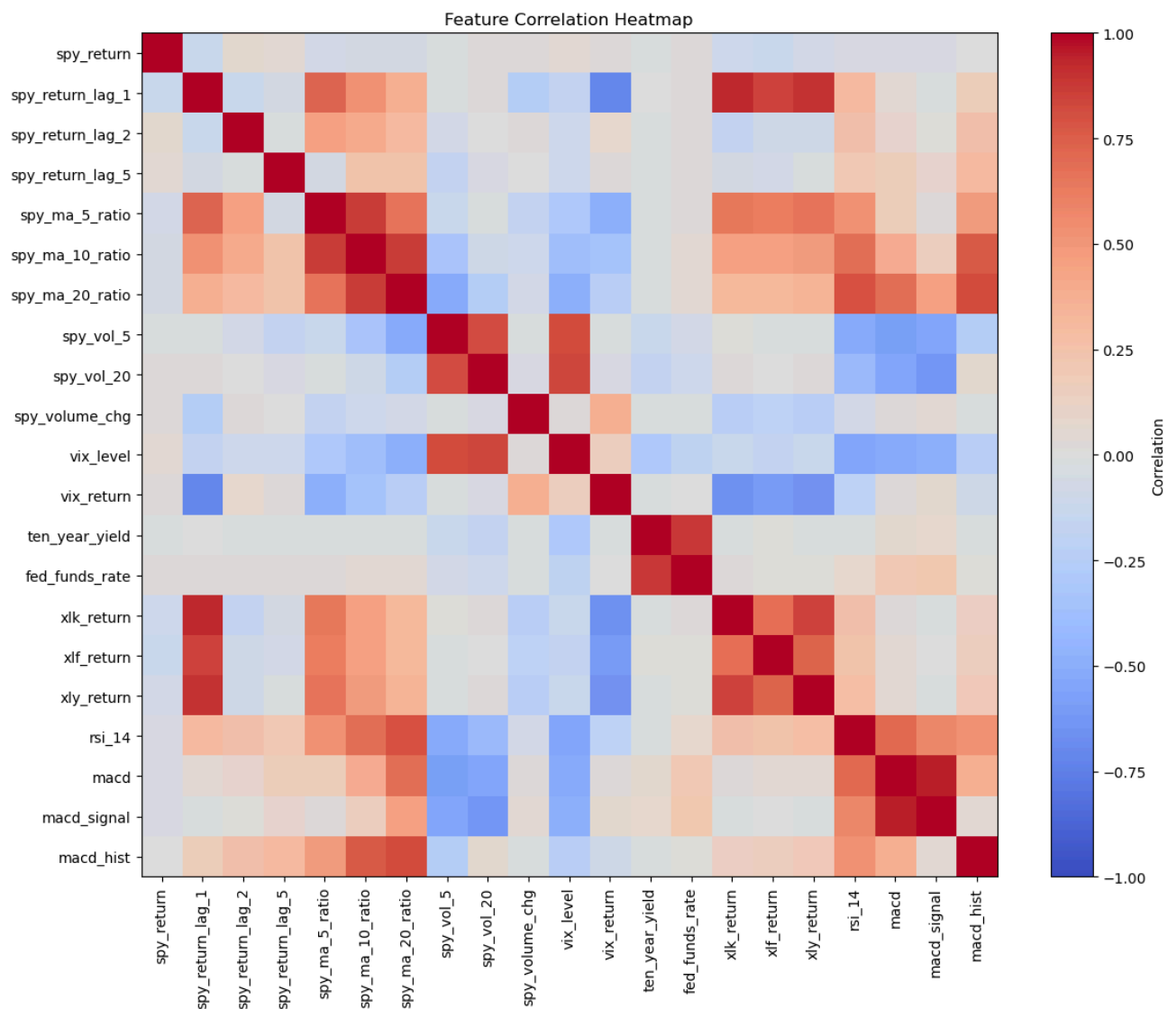


Figure 2. Correlation heatmap for the full modeling feature set.

Table 11. Top 10 Features by Absolute Correlation with Target

	abs_corr_with_target	corr_with_target
xlf_return	0.034613	0.034613
spy_return	0.033899	-0.033899
xlk_return	0.022353	0.022353
spy_volume_chg	0.016858	-0.016858
vix_return	0.016763	-0.016763
ten_year_yield	0.016382	-0.016382
spy_return_lag_5	0.015524	-0.015524
spy_return_lag_1	0.014261	0.014261
spy_vol_20	0.013993	-0.013993
fed_funds_rate	0.013937	0.013937

4. Methodology

4.1 Overview of Models and Validation Procedure

This project uses supervised machine learning classification models to predict the next-day return direction of SPY. The target variable equals 1 if the next-day SPY return is positive and 0 otherwise. All models use the same cleaned dataset, the same feature set, and the same target variable so that their results can be compared fairly.

We compare four models: Logistic Regression, Support Vector Machine, Random Forest, and K-Nearest Neighbors. These models represent different approaches to classification. Logistic Regression is a linear baseline model, SVM is a margin-based classifier that can capture nonlinear decision boundaries, Random Forest is a tree-based ensemble method, and KNN is a distance-based nonparametric method.

Because the observations are ordered by trading date, we use a chronological train-validation-test split instead of a random split. The training set is used to fit the models, the validation set is used to choose hyperparameters, and the test set is reserved for final performance evaluation. This prevents future observations from being used to predict earlier observations and better reflects a realistic financial forecasting setting.

To make the comparison fair, each group member should use the same `X_train`, `y_train`, `X_val`, `y_val`, `X_test`, and `y_test` created in the code below. Each model may have its own preprocessing and hyperparameter choices, but the final performance should be reported on the same test set. Model performance will be evaluated using accuracy, AUC, and the confusion matrix.

Feature scaling is handled model by model. Logistic Regression, SVM, and KNN should use `StandardScaler` inside a `Pipeline` because these methods are sensitive to feature scale. Random Forest does not require standardization because tree-based models split observations based on feature thresholds and are generally not affected by variable magnitude.

Table 12. Train / Validation / Test Split Summary

Period	Start Date	End Date	Observations	Up Days	Down/Flat Days	% Up
Train	2015-02-23	2020-12-31	1476	811	665	54.9%
Validation	2021-01-04	2022-12-30	503	255	248	50.7%
Test	2023-01-03	2024-12-31	502	288	214	57.4%

4.2 : Models

4.2.1 : Logistic Regression (Anna Chen)

Motivation

Logistic Regression is included as the baseline classification model. It is simple, interpretable, and commonly used for binary prediction problems. In this project, it estimates the probability that the next-day SPY return will be positive using the same lagged market, macro, sector, and technical indicators used by the other models. The model can be written as

$$P(Y_t = 1 \mid X_t) = \frac{1}{1 + \exp[-(\beta_0 + X_t^\top \beta)]},$$

so each coefficient describes how a one-standard-deviation change in a predictor is associated with the log-odds of an up day, holding the other predictors fixed.

Model description

We use a `LogisticRegression` model inside a `Pipeline` with `StandardScaler`. Scaling is necessary because Logistic Regression is sensitive to feature magnitude, and variables such as VIX level, interest rates, and daily returns are measured on different scales. The validation search considers different values of regularization strength `C`, penalty type (`L1` or `L2`), and `class_weight`. The best configuration is selected using validation AUC, which evaluates the ranking quality of predicted probabilities without depending on a fixed 0.5 classification threshold. After the best hyperparameters are selected, the final Logistic Regression model is refit on the combined training and validation periods before being evaluated on the held-out test set.

Why this method is appropriate

Logistic Regression is useful for this project because it provides a transparent linear benchmark against more flexible models such as KNN, Random Forest, and SVM. We do not fix `class_weight="balanced"` in advance; instead, both `class_weight=None` and `class_weight="balanced"` are compared on the validation set. This supports the modeling choice with empirical validation rather than assuming that class weighting is always appropriate. Because all predictors are standardized inside the pipeline, coefficient

magnitudes are comparable across features; the coefficient plot is therefore used as an interpretability diagnostic rather than as evidence of causal relationships. A limitation is that Logistic Regression assumes an approximately linear relationship between the predictors and the log-odds of an up day, so nonlinear effects and feature interactions may not be captured unless they are explicitly engineered.

1. Model

Table 13. Logistic Regression Class Weight Comparison

	Class Weight	Best Validation AUC	Best Validation Balanced Accuracy
0	None	0.533700	0.527200
1	balanced	0.534500	0.530100

Table 14. Logistic Regression Final Test Summary

Item	Value
Model	Logistic Regression
C	1.000000
Penalty	L2
Class Weight	balanced
Validation AUC	0.534500
Test Accuracy	0.452200
Test Balanced Accuracy	0.510600
Test Precision	0.622600
Test Recall	0.114600
Test F1	0.193500
Test AUC	0.538500
TN	194
FP	20
FN	255
TP	33

2. Visualizations

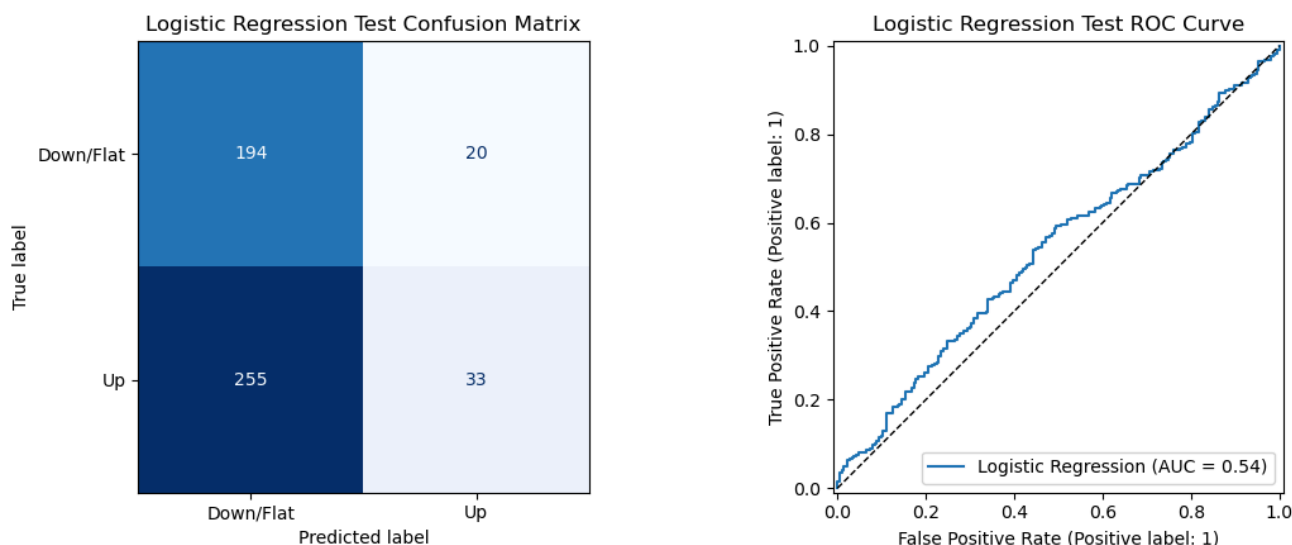


Figure 3. Logistic Regression confusion matrix and ROC curve on the test set.

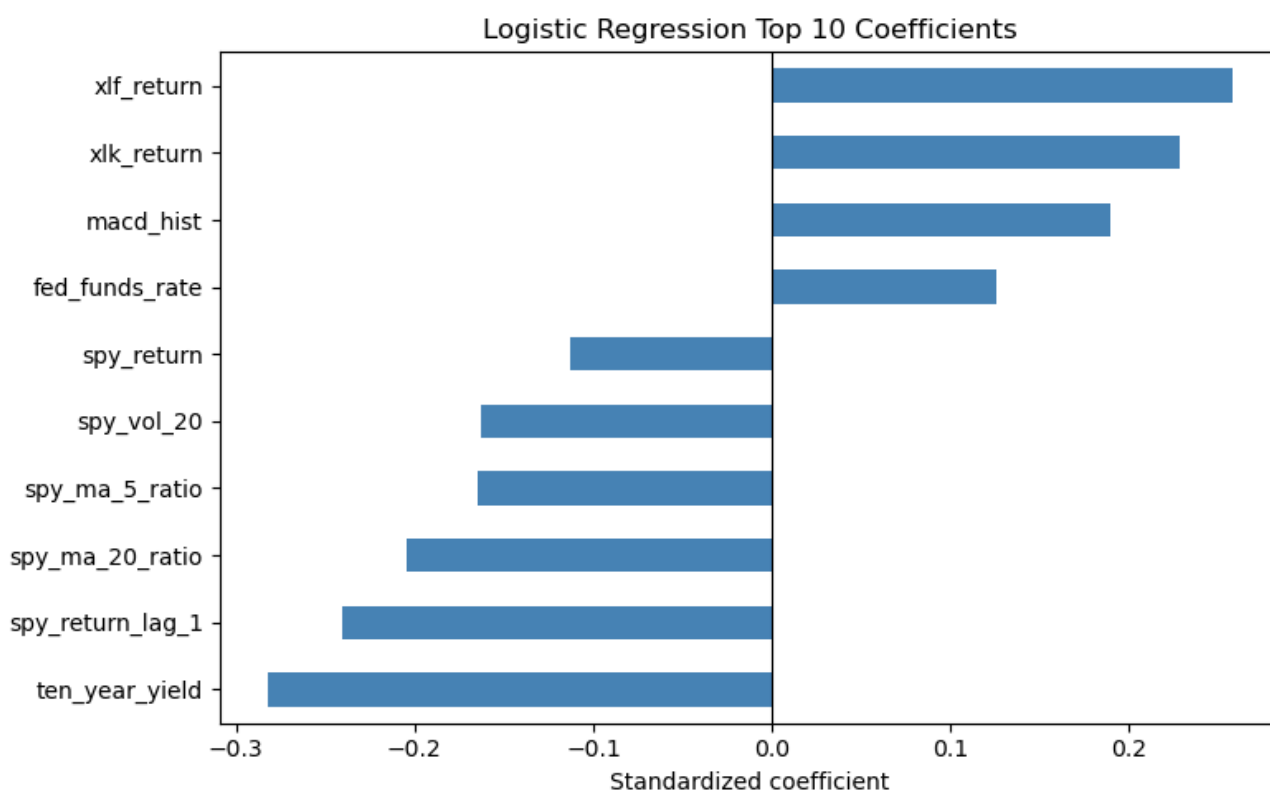


Figure 4. Top 10 Logistic Regression coefficients by absolute standardized magnitude.

Logistic Regression results interpretation

Table 14 and Figure 3 show that the final Logistic Regression model has a test AUC of 0.5385, which is slightly above the random-ranking benchmark of 0.50 but still indicates weak out-of-sample predictive signal. The test accuracy is 0.4522 and the balanced accuracy is 0.5106, so the model does not produce strong classification performance at the default threshold. However, its precision is 0.6226 while its recall is only 0.1146, meaning that when the model predicts an up day it is relatively selective, but it misses many actual up days. This pattern is

also visible in the confusion matrix, where the number of false negatives is large relative to true positives.

Figure 4 provides an interpretability check based on standardized coefficients. Because the features are scaled before fitting, the coefficient magnitudes can be compared across predictors. The largest coefficients identify the variables with the strongest linear association with the predicted log-odds of an up day in this fitted model. These coefficients should be interpreted as predictive associations within the model rather than causal effects. Overall, Logistic Regression provides a transparent benchmark and produces the highest test AUC among the current model set, but the magnitude of the AUC suggests that any next-day direction signal remains limited.

4.2.2 : KNN (Yue Lu)

Motivation

K-Nearest Neighbors is included as a nonparametric benchmark. Unlike Logistic Regression, KNN does not impose a linear relationship between the predictors and the probability of an up day. Instead, it classifies a new observation based on the historical observations that are most similar in feature space.

Model description

We use a `KNeighborsClassifier` inside a `Pipeline` with `StandardScaler`. Scaling is necessary because KNN relies on distance calculations, and unscaled variables such as VIX level or interest rates could otherwise dominate return-based features. The validation search considers different values of `n_neighbors`, distance weighting schemes, and distance metrics. The best configuration is chosen using validation performance before evaluating on the test set.

Why this method is appropriate

KNN is useful for this project because it can capture local patterns without specifying a parametric model. If days with similar volatility, momentum, and macro conditions tend to have similar next-day directions, KNN may identify those relationships. It also provides a contrast to the linear baseline and tree-based ensemble model.

1. Model

Best KNN Parameters:

```
{'n_neighbors': 51, 'weights': 'uniform', 'metric': 'manhattan'}
```

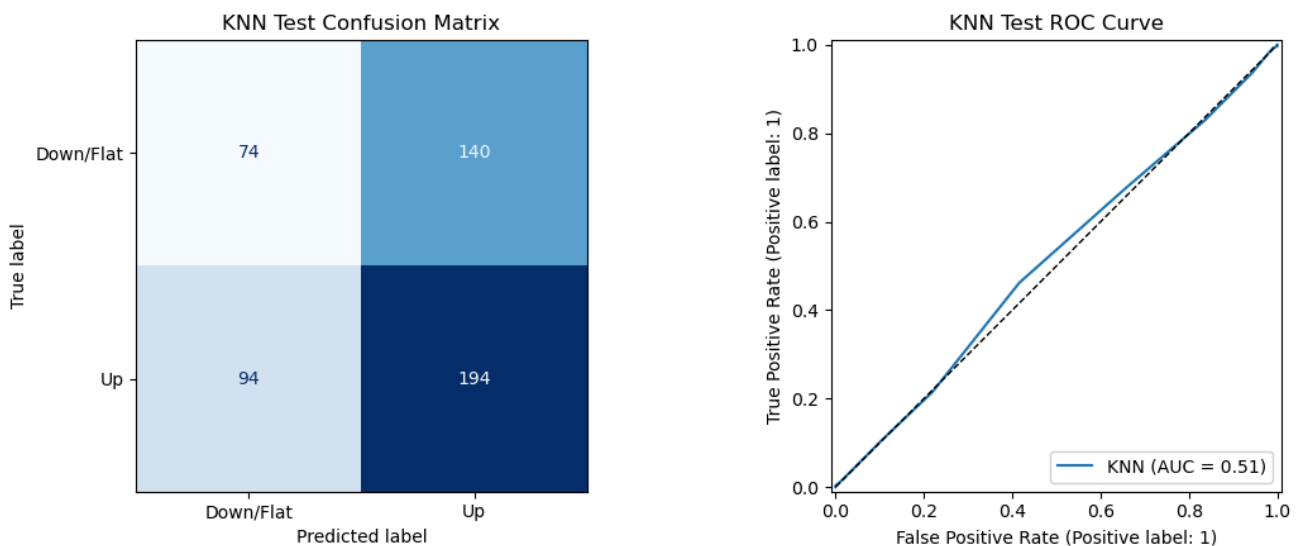
Table 15. Top 5 KNN Validation Results

n_neighbors	weights	metric	val_f1	val_auc
51	uniform	manhattan	0.581300	0.479900
51	distance	manhattan	0.580800	0.478300
41	uniform	minkowski	0.570900	0.484700
41	distance	manhattan	0.570500	0.490300
41	uniform	manhattan	0.567100	0.493300

Table 16. KNN Validation and Test Performance

sample	accuracy	balanced_accuracy	precision	recall	f1	auc
Validation	0.497000	0.495600	0.503300	0.600000	0.547400	0.495500
Test	0.533900	0.509700	0.580800	0.673600	0.623800	0.512200

2. Visualizations

**Figure 5.** KNN confusion matrix and ROC curve on the test set.

4.2.3 : Random Forest (David Zhang)

Motivation

Since next-day SPY direction prediction involves numerous features spanning from macroeconomic conditions to technical indicators, a feature set whose joint predictive structure is unlikely to be linear or additive. Random Forest captures nonlinear interactions among these variables automatically. It also requires no feature standardization, unlike logistic regression and SVM, which eliminates a potential source of leakage from the preprocessing step. For a noisy, short-horizon prediction task with over 1000 training observations, ensemble averaging across many bootstrapped trees meanwhile reduces prediction variance compared to a single decision tree.

Model description

Random Forest builds many decision trees on bootstrap samples of the training data and averages their predicted class probabilities. At each node, only a random subset of features is considered for splitting, which decorrelates the trees and reduces ensemble variance.

Hyperparameter	Function	Candidates
<code>n_estimators</code>	Number of trees: more trees, lower variance	200, 300, 500
<code>max_depth</code>	Maximum tree depth: limits overfitting	3, 5, 8, None
<code>min_samples_leaf</code>	Minimum observations per leaf: smooths decision boundary	1, 5, 10

Hyperparameter	Function	Candidates
max_features	Features considered per split: decorrelates trees	sqrt , 0.5
class_weight	Reweights minority class: addresses class imbalance	None , balanced
ccp_alpha	Post-growth pruning: simplifies trees	np.linspace(0.0, 0.02, 6)

Why this method is appropriate

In general, random Forest suits our task for three reasons. First, it requires no StandardScaler and eliminating a potential leakage risk present in logistic regression and SVM. It captures nonlinear interactions across the 21 heterogeneous features without manual specification. Besides, its ensemble structure reduces prediction variance in a high-noise, small-sample setting where daily equity return prediction is inherently difficult (Ansah, 2022). Lastly, the mild class imbalance of roughly 55% Up versus 45% Down days is addressed by including class-weight as a tuned hyperparameter, letting validation AUC determine empirically whether reweighting improves out-of-sample performance.

1. Data Processing

Table 17. Data Distribution

	Down/Flat	Up
Split		
Train	0.450500	0.549500
Validation	0.493000	0.507000
Test	0.426300	0.573700

The class balance table shows whether Up and Down/Flat days are evenly represented across the train, validation, and test periods. Because the class proportions are not exactly equal and may shift across time periods, the following Random Forest tuning grid includes class weight argument. This allows validation AUC to determine whether reweighting the less frequent class improves out-of-sample ranking performance.

2. Model

Table 18. Model Evaluation

model	best_params	validation_auc	test_accuracy	test_auc	tn	fp	fn	tp
Random Forest	{'ccp_alpha': np.float64(0.004), 'class_weight': None, 'max_depth': 3, 'max_features': 0.5, 'min_samples_leaf': 10, 'n_estimators': 200}	0.561700	0.426300	0.499000	214	0	288	0

Selected RF parameters: {'ccp_alpha': np.float64(0.004), 'class_weight': None, 'max_depth': 3, 'max_features': 0.5, 'min_samples_leaf': 10, 'n_estimators': 200}

After model setup, training, and train/test evaluation, the tuned Random Forest achieves a validation AUC of 0.5617, suggesting weak ranking ability during the validation period.

However, its test AUC drops to 0.4988 and its test accuracy is only 0.4263, indicating that the validation relationship does not generalize to the 2023-2024 test period. The result ultimately comes down to how the selected hyperparameters compound on each other rather than any single parameter being problematic in isolation.

With `max_depth=3`, each tree can only partition the feature space into 8 coarse regions across 21 features. It is already a very blunt instrument for a noisy prediction task. Stacking `min_samples_leaf=10` on top makes it worse, blocking many splits before they even happen. Then `ccp_alpha=0.004` prunes away what little remains, since most splits in a dataset where the strongest feature correlation is only $|r| \approx 0.07$ produce such marginal impurity reductions that they fall below the pruning threshold anyway. By the time a tree is fully grown, it is shallower and simpler than the already-shallow `max_depth=3` setting suggests.

With `class_weight=None` providing no additional push toward recovering either class, averaging 200 of these heavily regularized trees produces predicted probabilities that cluster in a narrow band between 0.45 and 0.50 for every single test day without once crossing the decision boundary. The model is not confidently predicting Down/Flat. Instead, it is essentially guessing on every observation, and that uncertainty just happens to land consistently on the wrong side of 0.5.

3. Visualizations

(1) Confusion Matrix & ROC curve

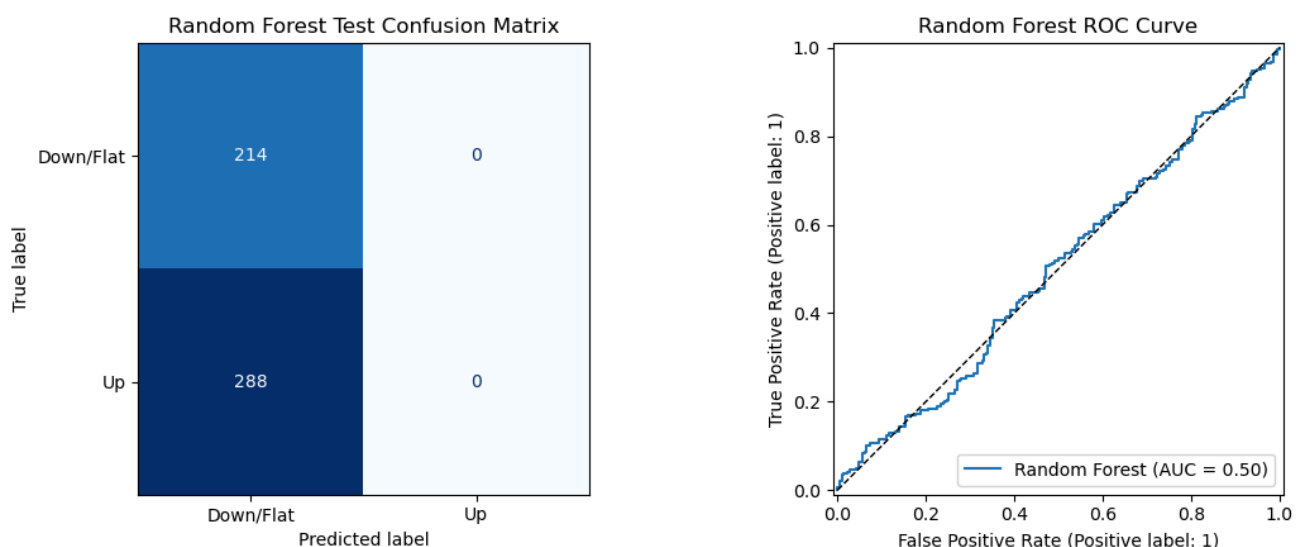


Figure 6. Random Forest confusion matrix and ROC curve on the test set.

The confusion matrix clearly reveals the pattern that the classifier successfully classified all 214 down days correctly but failed to classify all 288 up days, classifying all observations as Down/Flat. In no case was the Up prediction made; hence, there were zero true and false positives. This is also evident from the ROC curve plot where the ROC line is plotted just below the diagonal at an AUC score of 0.49, meaning the classifier scores positive cases a little less effectively than random. Together, the two plots give a similar view regarding the performance of Random Forest over the test period from 2023-2024 that it was unable to classify any signals to distinguish up from down days.

(2) Feature Importances

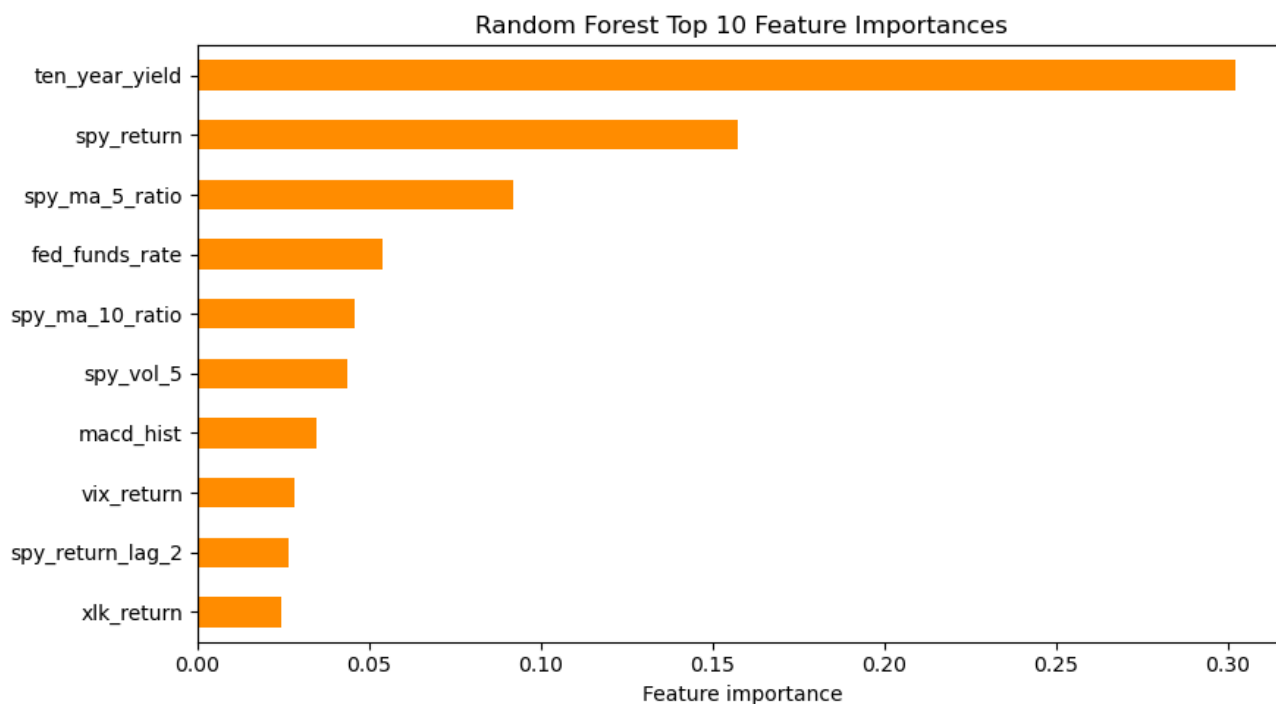


Figure 7. Top 10 Random Forest feature importances.

Ten-year-yield stands out dramatically, accounting for nearly 0.49 of total importance. This is likely a statistical artifact rather than genuine predictive power. The ten-year yield is a slowly varying level variable ranging from roughly 1% to 5% across the sample, giving it many distinct split points at each tree node. Random Forest's Gini-based importance tends to favor variables with many unique values, since they offer more opportunities to reduce impurity at each split. In contrast, daily return features are small and noisy, offering fewer clean split opportunities. The dominance of ten-year-yield here is therefore more a reflection of how the RF scores importance than evidence that yields genuinely drive next-day SPY direction.

(3) Probability Series

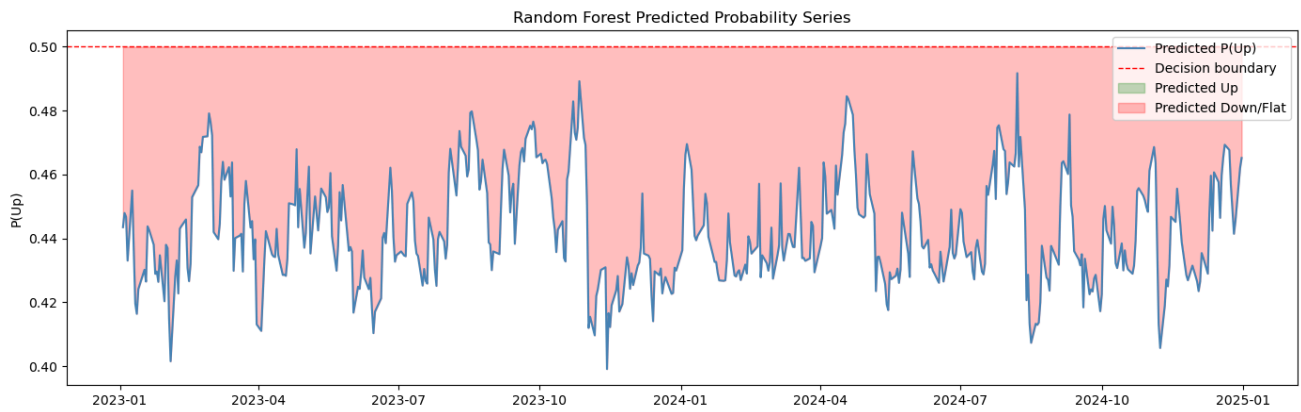


Figure 8. Random Forest predicted probability series on the test set.

The predicted probability series tracks the model's daily confidence that SPY will close up, plotted over the entire 2023–2024 test period. Values above 0.5 would signal a predicted Up day (green), while values below 0.5 signal a predicted Down/Flat day (red). As a response to changing market environment, there should be observable swings around the point in question. In reality, what is plotted is something quite the contrary: predicted probability lies completely squeezed in a small space from 0.45 to 0.50 without ever exceeding the decision boundary. Thus, for the whole two years, one gets a perfectly red plot with no green spots on it. Lack of true positives in the confusion matrix may be visually supported by the certainty of the model, which does not vary much despite the activity on the market. Furthermore, flatness of the line under discussion proves poor calibration of the model because it treats all trading days equally in terms of forecasting certainty.

4.2.4 : SVM (Jiayuan Li)

Motivation

SVM finds the maximum-margin hyperplane separating two classes in feature space, making it inherently resistant to noise — a desirable property when predicting daily return direction where signal-to-noise ratios are low. Its kernel trick allows the model to capture nonlinear structure without explicitly enumerating interaction terms.

Model description

We train a binary SVC with two candidate kernels:

- **Linear:** $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$, controlled by regularisation parameter C .
- **RBF:** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$, capturing local nonlinear geometry.

The signed distance from the decision hyperplane (`decision_function`) is used as the ranking score for AUC, avoiding the internal cross-validation that Platt scaling would introduce — which would violate the strict chronological separation required for time-series evaluation.

Hyperparameter tuning

Grid search is performed on the validation set (2021–2022); the configuration with the highest validation AUC is selected.

Hyperparameter	Candidates
kernel	linear , rbf
C	0.01, 0.1, 1, 10, 100
gamma	scale , auto (rbf only)
class_weight	None , balanced

Why this method is appropriate

- All features are standardised, satisfying SVM's scale-sensitivity requirement — with the scaler fitted on the training set only to prevent leakage.
- The max-margin objective generalises well in relatively high-dimensional settings (21 features, ~1,500 training observations).
- No distributional assumptions on returns are imposed.

1. Model

Table 19. Top 5 SVM Configurations by Validation AUC

kernel	C	gamma	class_weight	Val Acc	Val AUC	Val F1
linear	1.000000	scale	balanced	0.526800	0.535400	0.516300
linear	10.000000	scale	balanced	0.524900	0.530900	0.530500
linear	100.000000	scale	balanced	0.528800	0.530800	0.534400
linear	0.100000	scale	balanced	0.497000	0.526500	0.413000
linear	1.000000	scale	None	0.507000	0.524800	0.672800

2. Model Evaluation

Selected: kernel=linear, C=1, gamma=scale, class_weight=balanced, Val AUC=0.5354

Table 20. SVM Test-Set Performance

Metric	Test Score
Accuracy	0.571700
AUC	0.508900
Precision	0.585500
Recall	0.868100
F1 Score	0.699300

3. Visualizations

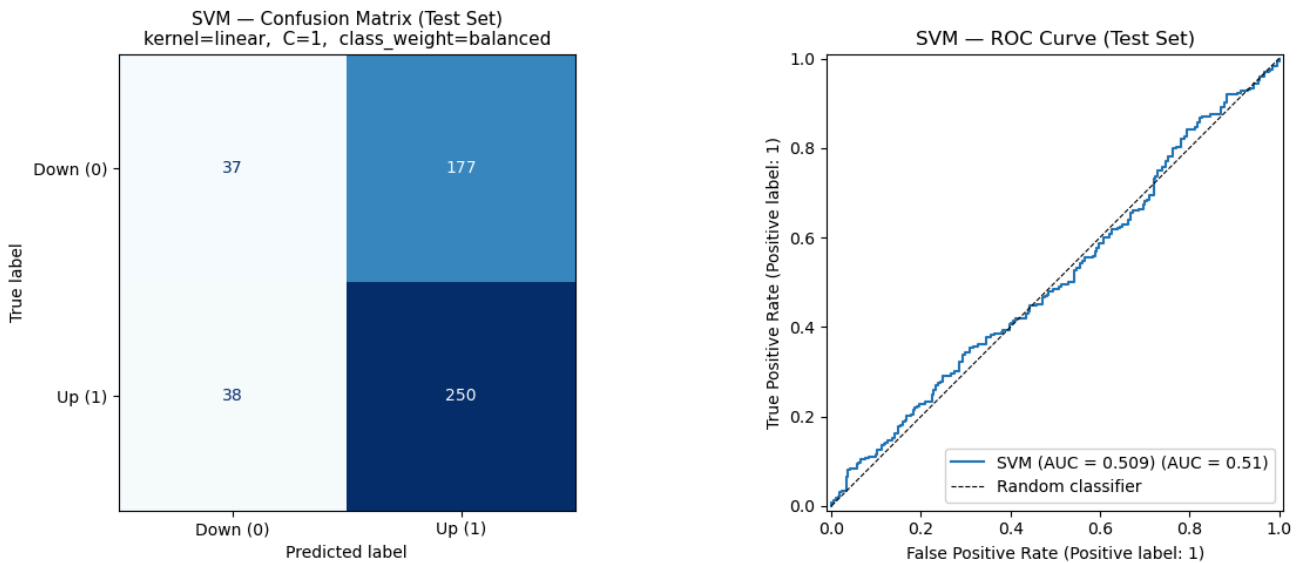


Figure 9. SVM confusion matrix and ROC curve on the test set.

4. SVM results interpretation

The final SVM model achieves the highest test accuracy and F1 score among the four models, with a test accuracy of 0.5717 and an F1 score of 0.6993. From the confusion matrix, the model correctly identifies many actual up days, with 250 true positives and only 38 false negatives. This explains its high recall of 0.8681. However, the model also produces 177 false positives, meaning that it often predicts an up day even when the market is actually down or flat. Therefore, the strong F1 score is partly driven by the model's tendency to classify many observations as positive during a test period where up days are more common.

At the same time, the test AUC is only 0.5089, which is very close to the random benchmark of 0.50. This suggests that although SVM performs well under threshold-based metrics such as accuracy and F1, it does not strongly rank up days above down or flat days. The selected model uses a linear kernel with `C = 1` and `class_weight = balanced`, indicating that the validation set favored a simpler max-margin boundary rather than a more flexible RBF kernel. Overall, SVM is the best classifier in terms of test accuracy and recall, but its limited AUC shows that the predictive signal remains weak.

5. Results

The results below are from the common 2023-2024 test set. Hyperparameters are selected using the 2021-2022 validation set, while the test set is held out for final comparison.

5.1 Evaluation Metrics

Since this project is a binary classification problem, we use accuracy, precision, recall, F1 score, and AUC to evaluate the models. Accuracy measures the overall percentage of correct predictions. Precision measures how often the model is correct when it predicts an up day. Recall measures how many actual up days the model captures. F1 score combines precision and recall, so it is useful for comparing the balance between them. AUC measures how well the model ranks up days above down or flat days based on predicted probabilities.

The current initial results show that next-day SPY direction is difficult to predict with standard machine learning models. SVM has the highest test accuracy and F1 score in this version, largely because it predicts many up days during the 2023-2024 test period, when the market had more up days than down days. Logistic Regression has the highest test AUC, but the value is still only slightly above 0.50. Random Forest has the strongest validation AUC within its own grid search, but this does not carry over to the test period, which suggests that the validation relationship may not be stable.

Overall, these results show that different metrics give different views of model performance. Accuracy and F1 score favor SVM because it captures many up days in the test period. AUC favors Logistic Regression because it ranks observations slightly better than the other models. However, since most AUC values are close to 0.50, none of the models shows strong predictive power. This is important because a model can have a reasonable accuracy or F1 score while still having weak probability ranking ability.

5.2 Main Quantitative Results

5.2.1 Model Performances

Table 21. Model Performance Metrics on the Test Set

Model	Validation AUC	Test Accuracy	Test Balanced Accuracy	Test Precision	Test Recall	Test F1	Test AUC
Logistic Regression	0.534500	0.452200	0.510600	0.622600	0.114600	0.193500	0.538500
KNN	0.495500	0.533900	0.509700	0.580800	0.673600	0.623800	0.512200
Random Forest	0.561700	0.426300	0.500000	0.000000	0.000000	0.000000	0.499000
SVM	0.535400	0.571700	0.520500	0.585500	0.868100	0.699300	0.508900

5.2.2 Confusion Matrix Counts

Table 22. Test-Set Confusion Matrix Counts by Model

Model	TN	FP	FN	TP
Logistic Regression	194	20	255	33
KNN	74	140	94	194
Random Forest	214	0	288	0
SVM	37	177	38	250

5.2.3 Model Hyperparameters

Table 23. Selected Model Hyperparameters

Model	Selected Parameters
Logistic Regression	{'model__C': np.float64(1.0), 'model__penalty': 'l2', 'model__class_weight': 'balanced'}
KNN	{'n_neighbors': 21, 'weights': 'uniform', 'metric': 'minkowski'}
Random Forest	{'ccp_alpha': np.float64(0.004), 'class_weight': None, 'max_depth': 3, 'max_features': 0.5, 'min_samples_leaf': 10, 'n_estimators': 200}
SVM	{'kernel': 'linear', 'C': 1, 'gamma': 'scale', 'class_weight': 'balanced'}

Logistic Regression has the highest test AUC among the four models, with a test AUC of 0.5385. This means it is slightly better than the other models at ranking up days above down or flat days. However, the AUC is still only a little above 0.50, so the improvement is limited. Its test accuracy is lower because it predicts fewer up days and misses many actual up days. This makes the model too conservative for this prediction task.

KNN has a test accuracy of 0.5339 and a test F1 score of 0.6238. Compared with Logistic Regression, KNN predicts more up days, so it has higher recall and a better F1 score. However, its test AUC is 0.5122, which is still close to random guessing. This means that KNN can classify some observations correctly, but it does not clearly separate up days from down or flat days.

Random Forest performs well during validation, but this result does not carry over to the test period. Its test AUC is 0.4988, which is almost the same as random guessing. This suggests that the patterns learned during validation may not be stable in the later test period. Since financial markets can change over time, a model that works well in one period may not perform well in another period.

SVM has the highest test accuracy and F1 score among the four models. Its test accuracy is 0.5717, and its test F1 score is 0.6993. This happens mainly because the SVM predicts many up days in the test period. Since the 2023-2024 test period had more up days than down days, this helps SVM achieve stronger classification results. However, its test AUC is only 0.5090, so the model is still not very strong at ranking observations.

Overall, SVM gives the best classification results based on accuracy and F1 score, while Logistic Regression gives the highest AUC. However, all models have AUC values close to 0.50, so the overall prediction problem remains difficult.

5.3 Visualizations

5.3.1 ROC curves

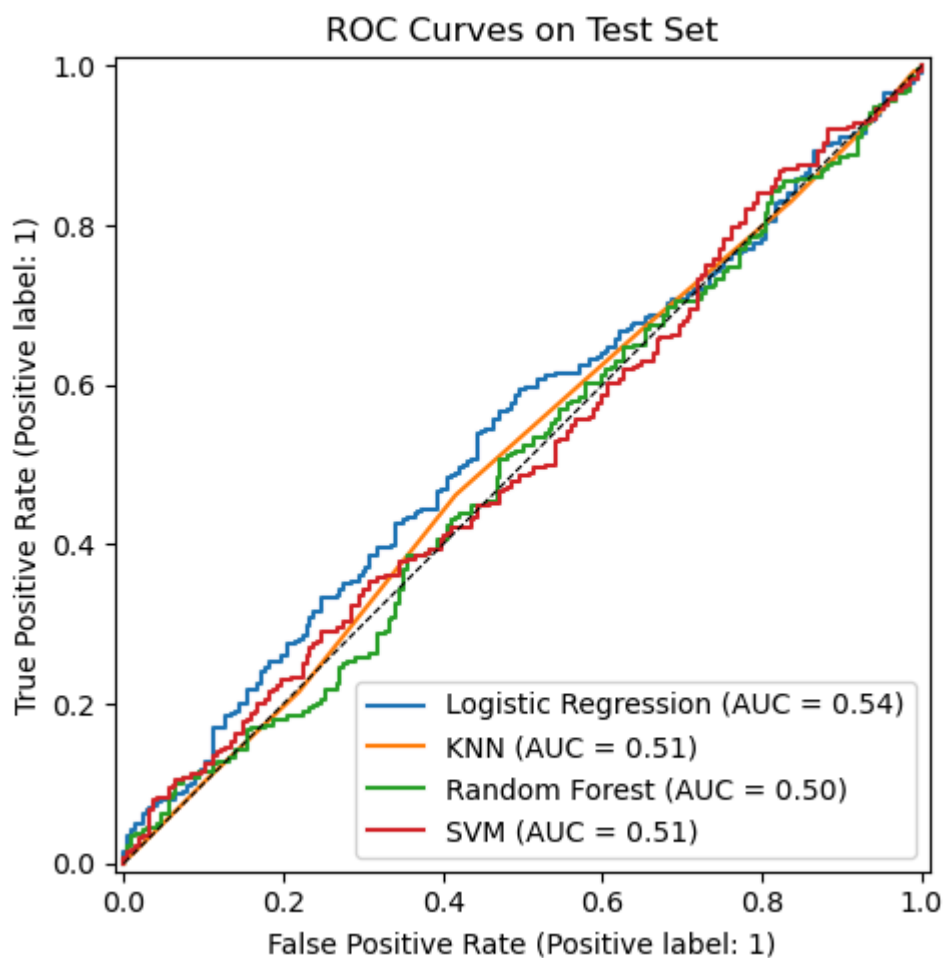


Figure 10. ROC curve comparison for all four models on the test set.

The ROC curves show that all four models are close to the diagonal benchmark line. This means that the models do not have strong ability to rank up days above down or flat days. Logistic Regression has the highest AUC, but the improvement over 0.50 is small.

5.3.2 Confusion Matrices

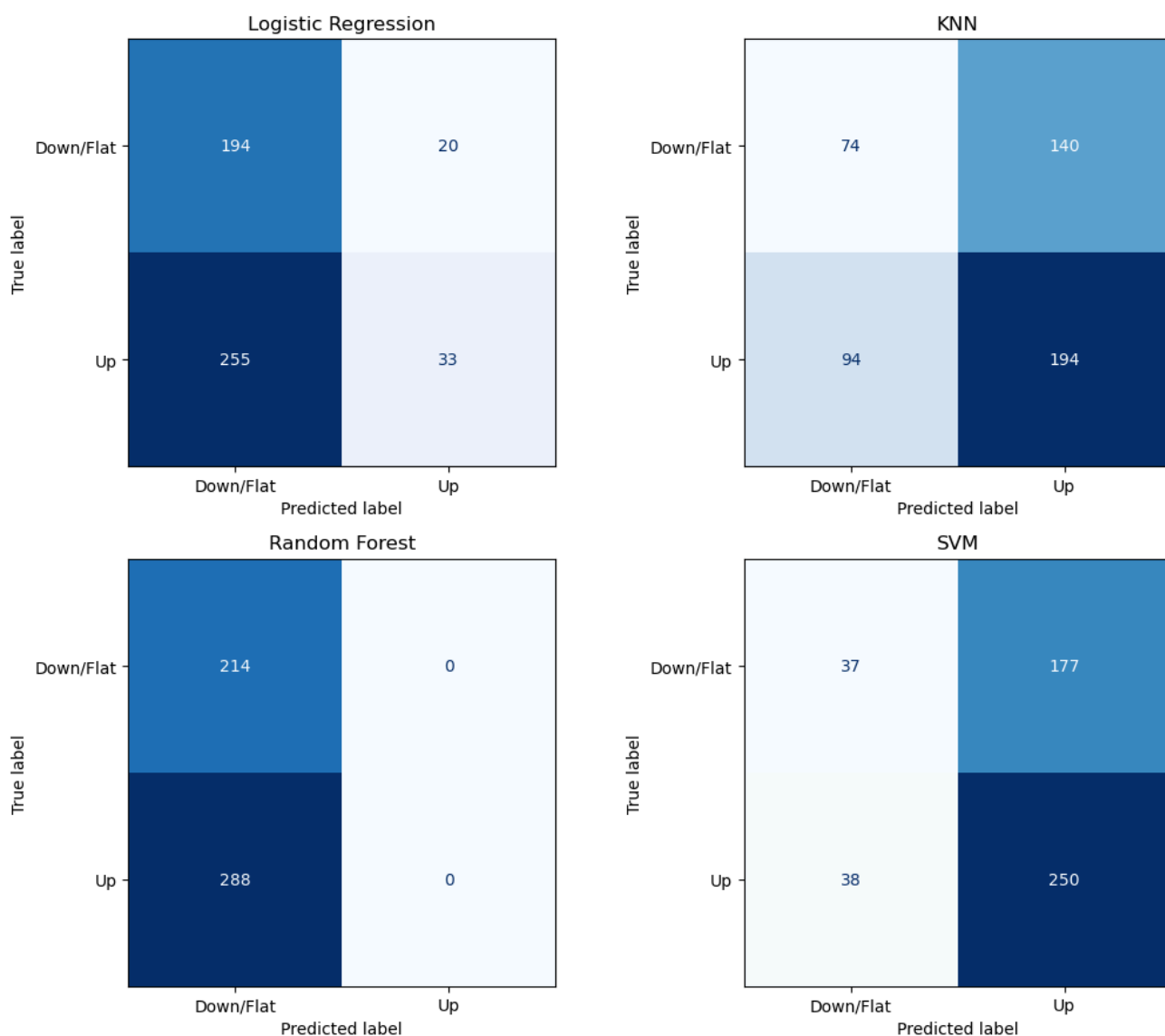


Figure 11. Confusion matrix comparison for all four models on the test set.

The confusion matrices help explain why the models have different accuracy and F1 scores. Logistic Regression predicts fewer up days, so it misses many actual up days. SVM predicts more up days, which increases its recall and F1 score. KNN also predicts many up days, but its overall performance is weaker than SVM. Random Forest does not perform well on the test set, even though it had stronger validation results.

Overall, the visual results are consistent with the numerical results. SVM performs best in classification metrics such as accuracy and F1 score, but all models still have weak AUC values. This supports the idea that next-day SPY direction is hard to predict with the current features.

6. Discussions and Conclusions

In this project, we used several machine learning models to predict the next-day return direction of SPY (market indicators, volatility measures, sector returns, interest rate variables). Among all the models, SVM had the best overall performance. This may be because SVM tries to find a separating boundary between positive and non-positive return days, which can work better when the data is noisy and the signals are weak. However, even though SVM performed the best, its improvement over the benchmark was still small. Because of this, we do not think the result shows that SPY direction can be reliably predicted. Instead, it suggests that the model was only able to capture a limited amount of useful information.

From a financial perspective, this makes sense because short-term stock market movements are very hard to predict. Daily returns usually contain a lot of noise, and many price changes are caused by unexpected news, changes in investor sentiment, macroeconomic shocks, or random market movements. Although variables such as volatility and interest rates may provide some information, they are not enough to explain most next-day movements. SPY is also a very liquid and widely traded ETF, so public information is likely reflected in prices quickly. This is consistent with the Efficient Market Hypothesis, which suggests that asset prices already reflect available information and that short-term return prediction is limited (Fama, 1970).

The main practical implication is that our models do not provide strong evidence of a reliable trading signal. Even though SVM performed better than the other models, the improvement was too small to conclude that it could be used to make profitable trading decisions. In real markets, weak prediction results would become even less useful after considering transaction costs, bid-ask spreads, slippage, and risk management. This study also has some limitations. We only used historical public variables, used a fixed train-validation-test split, and evaluated the models mainly with classification metrics instead of actual trading performance. Future work could include richer data (news sentiment, macroeconomic announcements, intraday data, rolling-window testing). However, any improvement should still be judged carefully by whether it is stable out of sample, not just by a small increase in accuracy.

Appendix. Reproducibility

The notebook can be reproduced by running all cells from top to bottom in the configured course environment. The data are downloaded from Yahoo Finance using `yfinance`; the Federal Funds Effective Rate is loaded from `FEDFUNDS.csv` when available, with a FRED URL fallback. The train-validation-test split is chronological and uses the same dates throughout the notebook.

Main software packages used in this project are Python, NumPy, pandas, matplotlib, yfinance, ta, and scikit-learn. Random seeds are set where applicable to make model fitting more reproducible, although small differences may still occur across package versions or data vendor updates.

Table 24. Software Package Versions

package	version
Python	3.13.9
numpy	2.3.5
pandas	2.3.3
matplotlib	3.10.6
yfinance	1.4.1
ta	not reported
scikit-learn	1.7.2

References

Ansah, K., Denwar, I. W., & Appati, J. K. (2022). Intelligent Models for Stock Price Prediction. *Journal of Information Technology Research*, 15(1), 1–17. <https://doi.org/10.4018/jitr.298616>

Aurélien Géron. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. “O’Reilly Media, Inc.”

Fama, E. (1970). Efficient Capital markets: a Review of Theory and Empirical Work. *The Journal of Finance*, 25(2), 383–417. <https://doi.org/10.2307/2325486>

Federal Reserve Bank of St. Louis. (2024). Federal funds effective rate. FRED. <https://fred.stlouisfed.org/series/FEDFUNDS>

Jiao, Y., & Jakubowicz, J. (2017). Predicting stock movement direction with machine learning: An extensive study on S&P 500 stocks. 2017 IEEE International Conference on Big Data (Big Data). <https://doi.org/10.1109/bigdata.2017.8258518>

Rodriguez, F. S., P. Norouzzadeh, Anwar, Z., E. Snir, & Rahmani, B. (2024). A machine learning approach to predict the S&P 500 absolute percent change. *Discover Artificial Intelligence*, 4(1). <https://doi.org/10.1007/s44163-024-00104-9>

Yahoo Finance. (2024). SPY, XLK, XLF, XLY, VIX, TNX historical data. <https://finance.yahoo.com>